

Name: Khadija Tul Kubra

Roll #: BITF22M025

Project Documentation

AI Chatbot for Social Engineering Attack Detection

1. Introduction

1.1 Project Overview

The objective of this project is to develop an **Artificial Intelligence (AI) Chatbot** capable of detecting messages that are indicative of **Social Engineering Attacks** (such as phishing or smishing) within common communication platforms (like SMS or chat applications).

I have developed a **Full Stack application** comprising an interactive frontend, a high-performance backend API, and a robust Machine Learning prediction model.

1.2 Problem Statement

Social Engineering attacks remain a prevalent method for cybercriminals to compromise security. These attacks exploit human psychology and emotions to trick users into divulging confidential information. The sheer volume and clever phrasing of these messages make manual detection challenging.

Solution is a web-based chatbot that takes a message input from the user. This input is processed using **Natural Language Processing (NLP)** techniques and then analyzed by a trained **Logistic Regression** model to predict whether the message is '**Safe**' (**HAM**) or '**Unsafe**' (**SPAM/Attack**).

2. Technology Stack

Component	Technology	Role
Frontend (UI)	Next.js (React)	Handles the User Interface and Chatbot interaction for input and result display.
Backend/API	FastAPI (Python)	High-speed API endpoint responsible for receiving data from the frontend and managing the ML model prediction process.
NLP & ML	Scikit-learn	Used for Data preprocessing, TF-IDF Vectorization , and Model Training/Prediction.
Server	Uvicorn	The ASGI server used to run the high-performance FastAPI application locally.

3. Machine Learning Pipeline

3.1 Dataset & Manipulation

- **Source:** Kaggle SMS Spam Collection Dataset.
- **Content:** Consists of thousands of real-world messages labeled as either 'ham' (safe) or 'spam' (unsafe/malicious).
- **Manipulation & Preprocessing:** To better fit the social engineering context and ensure model effectiveness, the data underwent specific changes:
 - **Text Cleaning:** Removal of stopwords (e.g., 'a', 'the'), punctuation, and special characters.

- **Label Interpretation:** The 'Spam' label was interpreted as 'Unsafe/Attack' and 'Ham' as 'Safe' for project context.

3.2 Data Vectorization (NLP)

- **Tool:** **TfidfVectorizer** (from scikit-learn).
- **Explanation:** Before training, text data must be converted into numerical format. **TF-IDF (Term Frequency-Inverse Document Frequency)** assigns a numerical weight to each word. This weight reflects the importance of a word in a specific message relative to the entire dataset, helping the model identify words commonly associated with attacks.

3.3 Model Training and Selection

- **Model:** **Logistic Regression**
- **Rationale for Selection:**
 - **Efficiency:** Highly efficient and fast for text classification tasks, making it ideal for a real-time prediction API.
 - **Simplicity:** It's a linear model that provides good baseline accuracy and is easy to interpret.
- **Output:** The model processes the vectorized message and returns a binary prediction: **0 (Safe/HAM)** or **1 (Unsafe/SPAM)**.

4. Project Architecture

The entire system operates in a three-tier structure:

1. **Frontend (Next.js):** Captures the user's message and sends it as a **POST** request to the Backend API.
2. **Backend (FastAPI):**
 - Hosts the **/predict** API endpoint.
 - Loads the pre-trained **ML Model** and **TF-IDF Vectorizer** objects into memory.
 - Receives the message, processes it using the Vectorizer, gets the prediction from the Model, and returns the result (**Safe / Unsafe**) back to the Frontend.

3. **ML Layer (Scikit-learn):** The core prediction engine, managed and served by the FastAPI backend.

5. Features

1. **Real-time message prediction:** Chat interface where users type a message and get an immediate prediction.
2. **Safe / Unsafe classification:** Messages are classified based on social engineering patterns.
3. **Rule-based auto labelling:** Improves ML accuracy by detecting common unsafe patterns (URLs, urgent keywords, OTP, account requests).
4. **Synthetic augmentation:** Adds examples to increase the training dataset for better prediction accuracy.
5. **Download chat history:** Users can save conversation as a `.txt` file.

6. Running the Application (Local Setup)

This section details the steps required to set up and run the project locally.

6.1 Project-folder

project-folder/

├─ Chatbot info/

| └─ ml_backend/

```

├─ dataset_cleaned.csv

├─ main.py      # FastAPI backend code

├─ app.py # ML training script

├─ se_detector_model.pkl

├─ vectorizer.pkl

|  └─ app/      # Next.js frontend app

      └─ api/

            └─ chat-stream/

                  └─ route.ts/

├─ README.md

```

6.2 Installation Instructions

1. Clone the repository:

Gitclone

https://github.com/Khadija-tulkubra/AI_Chatbot-for-social-engineering-attack-detection

cd Chatbot Info

2. Backend Setup:

cd ml_backend

pip install -r requirements.txt # FastAPI, scikit-learn, pandas, joblib

uvicorn main:app --reload --port 8000

3. Frontend Setup:

```
npm install
```

```
npm run dev
```

4. Open your browser and access frontend at <http://localhost:3000>.
5. Chat with the bot and see safe/unsafe predictions.